

Automated Software Testing

Course Project

Louis Meller / lmeller@student.ethz.ch

Kevin Collins / collinsk@student.ethz.ch

Zurich, May 2026

Contents

1	Introduction	1
2	Design and Implementation	1
2.1	Scheduling	2
2.2	Mutation Rules	2
2.3	Corpus Handling	3
2.4	SQLite Integration	4
3	Results and Evaluation	4
3.1	Bug-finding Capabilities	5
3.2	Query Generation Characteristics	5
3.3	Coverage	7
3.4	Performance Evaluation	7
4	Discussion and Limitations	8
4.1	Configuration	8
4.2	Future and Optimization	9
5	Conclusion	9
6	Disclaimers	11
6.1	LLM Usage	11
6.2	Code Availability	11
	References	12
	Appendix	13
A	Coverage Metrics for Long Run Profile	13
B	Throughput Benchmarks	14
C	SQL Keyword Coverage Data	15

1 Introduction

Database management systems are complex software systems used to store, retrieve, and manage data effectively. As they are a critical part of a variety of software systems, it is highly important to ensure their correctness and robustness. Although it is clear that these systems need to be tested extensively, this can be challenging due to the large input space associated with SQL grammars. A testing technique that can work well in this scenario is fuzzing, which can be used to automatically generate a large number of test cases to discover crashes, memory safety violations, and logical inconsistencies. In this paper, we will evaluate the development of a coverage-guided grey-box fuzzer built to find bugs in database engines, specifically a patched version of SQLite 3.39.4, using the stable SQLite version 3.51.1 for differential testing. Our evaluation demonstrates that while the system achieves high throughput and identifies multiple bugs, it eventually encounters structural coverage plateaus

Our system combines mutation-based fuzzing with an evolutionary scheduling approach that prioritises queries according to their observed contribution to code coverage. Queries are represented internally as Abstract Syntax Trees (ASTs), enabling structured mutations and more effectively maintaining syntactic correctness than purely textual approaches. Candidate queries are primarily evaluated using fitness metrics derived from coverage expansion and execution characteristics. To support scalability during extended executions, the system also incorporates asynchronous batch execution, differential testing oracles, corpus caching, and periodic garbage collection strategies.

The system has been implemented as a Rust library that separates the genetic algorithm, mutation framework, and execution infrastructure, allowing the construction of different SQL fuzzers in either Rust or Python. We evaluate the system using both short- and long-run fuzzing configurations and analyse its performance characteristics. Then, we conclude by discussing the observed limitations in coverage growth and mutation effectiveness.

2 Design and Implementation

Our fuzzer is a grey-box mutation tester based on a provided seed corpus and coverage information from the Unit under Test (UUT). The core design uses an evolutionary approach that mutates a continuous population of SQL queries by applying sequences of mutation rules, favouring entries and rules that lead to the discovery of new edges in the UUT. Based on this coverage information, as well as runtime data, mutated entries are added back into the population or ignored. Unfit or redundant entries are periodically culled to limit the total population size.

Coverage is collected during execution using the `Clang -fsanitize-coverage=trace-pc-guard` option and a pool of shared memory buffers. This allows efficient collection of coverage data without having to invoke additional tools, such as `gcov`. The buffer pool automatically limits the number of simultaneous subprocesses, as buffers can only be shared with one child at a time and must be cleared after execution.

2.1 Scheduling

Parents are scored based on various metrics, including coverage, execution time, size, and acceptance ratio, which allows tuning the structure of produced SQL queries. Initial scores are calculated using a weighted moving average of the parents' own metrics and the results of their direct children, as the quality of the children derived from some entry is what matters. The recorded metrics are scaled by a continuously growing inflation scaler to weight recent events more heavily than historical data, preventing the scheduler from becoming stuck on queries that performed well only in the early stages. The final fitness is calculated using an adapted UCB1 scoring function [1]. This scoring scheme allows balancing exploitation based on the recorded metrics with exploration of entries that have not been used for a long time. The weights for the metrics that contribute to the fitness score are configurable per scheduler.

The same scoring system applies to the mutation rules themselves, as rule performance is expected to vary at different stages of the fuzzing run [2, p. 12]. Corpus scheduling is performed through Roulette-wheel selection with stochastic acceptance [3], which allows memory-efficient $O(1)$ probabilistic selection of the next corpus entry. On the other hand, rule scheduling uses a `PriorityQueue` to apply the top N rules per batch, since both N and the number of scheduled rules are small. Although rule scheduling decisions are made individually for each item in a batch, the schedulers do not receive updates during this time, meaning that the same sequence of rules will be scheduled for each entry in a batch.

2.2 Mutation Rules

We implemented a large set of mutation rules with varying degrees of generality. Initially, we implemented targeted rules, such as *TypeCast*, which mutates a `Value` to a `CAST(Value AS ...)`. However, it became clear that the SQL grammar is too large to cover with such specific approaches. Consequently, we took inspiration from Nautilus [2, p. 5] to implement more generic mutation rules:

- **SpliceIn:** Splices a random slice of statements from one parent into the child.
- **RecursiveExpand:** Recursively expands an expression node up to depth N . To

prevent exponential subtree growth, subtrees are truncated before mutation.

- **TreeMutate:** Shuffles nodes and injects null values into arbitrary nodes using JSON serialization.
- **ArbitraryGenerator:** Generates a random node and replaces one of the child’s nodes with it.

Implementing these required us to use JSON serialization to circumvent the strict AST typing and write and add support for crates like **Arbitrary** to our fork of **sqlparser-rs**.

To counteract the explosion in query size resulting from mutation rules such as *SpliceIn* and *RecursiveExpand*, we implemented rules working to reduce query size:

- **SpliceOut:** Removes a random slice of Statements from the child.
- **HoistExpr:** Collapses up to N layers of subtree nesting by moving the leaf node up.

Since query size is part of the fitness score of entries and rules, the scheduler can regulate how often query size reducing and increasing rules are applied to balance query size versus coverage found.

The inclusion of *ArbitraryGenerator* means that the system is not strictly limited to mutation, as this rule could theoretically build a corpus from scratch. However, relying completely on random generation is inefficient, as it may require significant time before important structures, such as valid **CREATE TABLE** statements, become sufficiently abundant in the corpus to allow for deeper exploration of the UUT.

As query size reduction and random generation are primarily useful for long runs, we implemented a specific “short-run” rule-set that excludes them. The “long-run” rule-set utilises an outer scheduler over *Repeat* rules, which repeat some rule up to N times, to allow the scheduler to adapt aggression over time, and multiple shared inner schedulers for the rules listed above.

2.3 Corpus Handling

Queries are stored as serialized ASTs in binary blobs, with SQL strings being saved lazily. The corpus handler caches the top N queries by fitness and recent access and the 2048 newest queries in RAM. Since cache misses involve reading from disk, deserializing the data and allocating space for the new entry, we try to minimise cache misses as much as possible. Thus the cache for the highest scoring entries grows dynamically, based on the recent cache miss rate. For short runs, a pure in-memory alternative is available. Periodic

garbage collection (GC) is triggered every 2000 epochs and if recent corpus growth exceeds 20%. During GC, queries are marked for removal unless they meet specific criteria:

1. Part of a greedy set that spans the total coverage discovered.
2. Part of 2048 new queries randomly selected via the SimHash Hamming distance of the AST.
3. Among the top N fittest queries.

Queries are removed from the corpus and scheduler, once they have accumulated 5 marks. Tuning these numbers allows one to adjust the corpus growth rate and steady state size.

2.4 SQLite Integration

The generated batches are executed asynchronously in 16 persistent workers. Workers are restarted on crashes, hangs, and every 50 executed queries to prevent state leakage. Results are sent to the engine orchestrator for corpus management and to 8 asynchronous oracle workers for differential testing against the provided binary of SQLite version 3.51.1. Roughly half as many oracles as executors can keep up with the executors, since queries resulting in runtime errors do not need to be tested by the oracle and oracle execution is faster due to not being instrumented with `-fsanitize-coverage=trace-pc-guard` and ASAN. Using Python’s `asyncio` framework allows us to easily balance multiple concurrent SQLite subprocesses across the oracles, executors and query generation.

The genetic algorithm is implemented as a Rust crate with Py03 Python bindings, allowing modular SQL fuzzer construction in Python or Rust.

3 Results and Evaluation

Disclaimer: To fulfil the requirement of generating a fixed number of queries, the engine’s internal corpus logic was bypassed to save all generated queries directly. This behaviour can be enabled by passing `eval_requirement true` to our CLI. Consequently, stopping after N generations results in an internal corpus size of $< N$. These short runs allow for the disabling of dynamic corpus handlers and periodic tasks like garbage collection. Additionally, the corpus scheduler is expected to act similarly to a weighted random selector over new entries during growth phases, as the fitness of new entries is heavily inflated.

To calculate the syntax error rate, keyword distribution, and coverage, we used the “short run” configuration (triggered by stopping at $\leq 20,000$ queries). Data for the “long run”

profile, which includes rules like *ArbitraryGenerator* and *SpliceOut* as well as chore tasks like GC and cache resizing, can be found in the Appendix.

3.1 Bug-finding Capabilities

During our evaluation, Address Sanitizer (ASAN) identified three distinct crashing inputs in the instrumented SQLite binary and three logic bugs. Initial triage indicates that these represent at least five unique root causes.

“Bug 1” and “Bug 2” appear to be closely related and could potentially be a case of duplication, as when the respective queries were executed on a fresh SQLite instance, both queries result in the same error (an AddressSanitizer negative-size-param failure in memcmp. We hypothesise that state leakage from a previous execution likely changed the exact execution path of “Bug 1”, leading to different behaviour from “Bug 2”.

On the other hand, “Bug 3” is distinct. It is triggered by an incomplete ‘rtree’ virtual table declaration, and this points to a completely separate root cause. All bugs we found are memory safety-related crashes, including two **SEGV** crashes and an ASAN termination.

“Bug 4”, “Bug 5” and “Bug 6” are different logic bugs, where the output of the oracle and the UUT differ for the same query input.

3.2 Query Generation Characteristics

The distribution of the top 30 most frequently generated SQL keywords, illustrated in Figure 1, demonstrates the fuzzer’s ability to maintain structural validity. While foundational keywords such as **CREATE** and **TABLE** have up to 99.6% frequency and are present in almost all queries, the high frequency of relational and analytical keywords like **JOIN** (27.9%), **GROUP** (21.6%), and **CAST** (68.4%) indicates deep exploration of the SQL grammar.

As shown in Table 10, most keywords appear less than once per query on average. However, a few notable outliers are clearly visible. **CAST** and **AS** occur 7.7 and 6.6 times per query respectively. This can be attributed to the *TypeCast* rule, which is part of the “short run” profile and which adds type casts to the mutated queries. **CREATE** and **TABLE** are also occurring between 2 and 3 times per query on average, likely because many queries in the seed corpus have multiple **CREATE TABLE** statements.

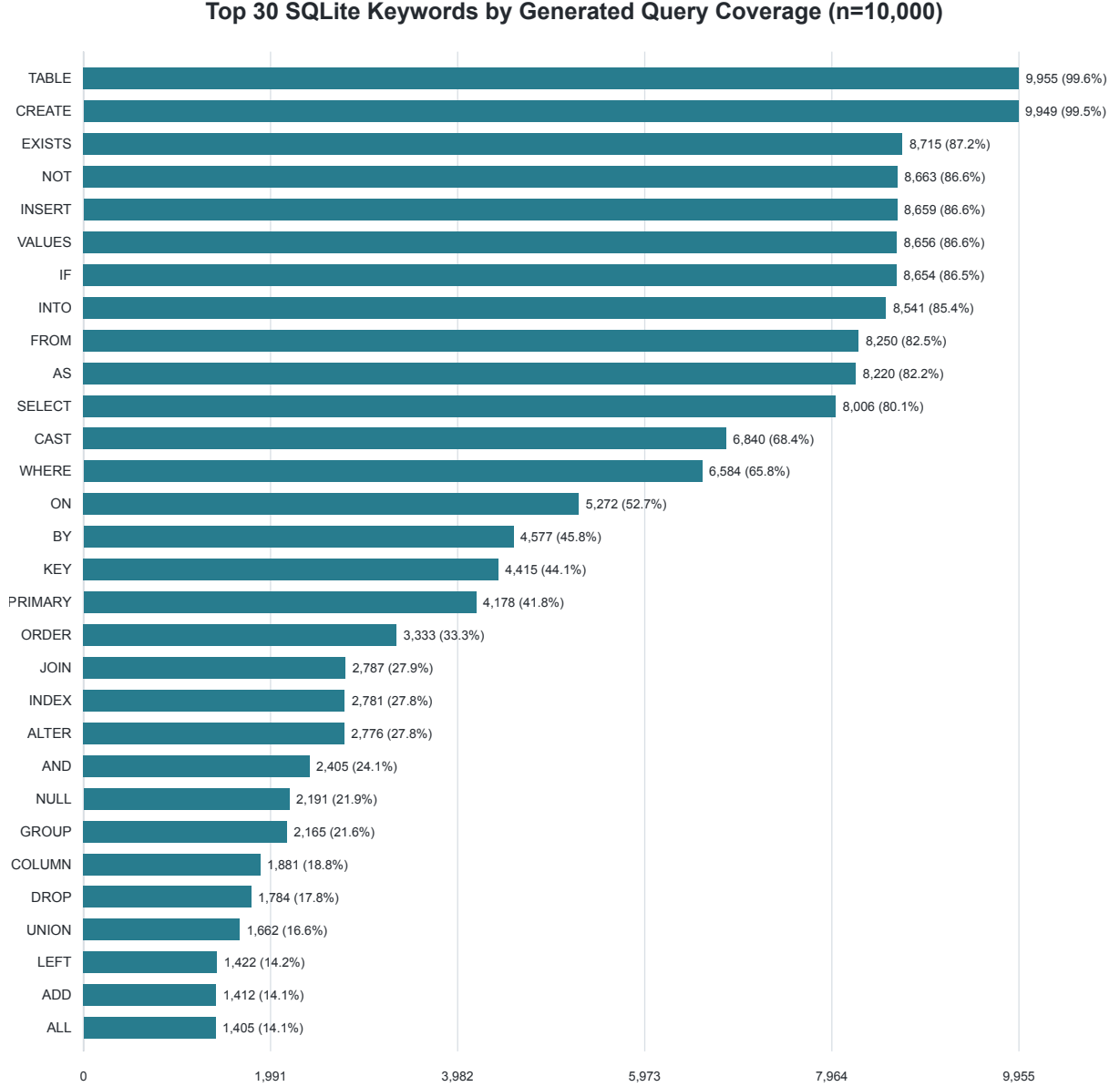


Figure 1: Top-30 most frequently used SQL keywords (Short Run Profile).

Notably, the keyword frequencies in Figure 1 suggest the successful generation of complex *semantic units* rather than isolated tokens. The high co-occurrence of sequences such as `CREATE TABLE IF NOT EXISTS`, `CAST ... AS` or `SELECT ... FROM ... WHERE` confirms that the AST-based mutation rules effectively preserve the logical relationship between SQL nodes.

With regards to query validity, we also found a relatively high rate of valid *semantic units*. On average, using the short profile, 4830 of the 10,000 queries are valid. More detailed information about the invalid query rate in the first 10000 generated queries across both short and long profiles can be found in Table 2.

3.3 Coverage

We distinguish between two types of coverage metrics: standardised code coverage via `gcov` and internal edge coverage reported by our fuzzer via

`Clang -fsanitize-coverage=trace-pc-guard`. Table 1 presents the `gcov` results for the 10,000 generated queries using the “short run” profile.

Table 1: Standardised `gcov` Metrics (Short Run Profile, 10,000 Queries).

Metric	Coverage %	Items Covered	Total Items
Lines	38.7%	31,103	80,277
Functions	44.1%	1,895	4,301
Branches	30.9%	16,565	53,691

To evaluate the impact of different mutation strategies, we compared the “short run” and “long run” profiles over the first 10,000 generated queries. As shown in Table 2, the *Short* profile outperforms the *Long* profile in both coverage and runtime error rate on this benchmark. The coverage for the “long run” profile as reported by `gcov` can be found in Table 4.

Table 2: Comparative Profile Performance (Internal Edge Coverage, 10000 Queries) across 20 runs.

Configuration	Internal Edge Coverage %	Invalid Query Rate
Short Profile	19.161% (± 0.254)	0.517 (± 0.049)
Long Profile	18.541% (± 0.215)	0.677 (± 0.024)

The “Long run” profile utilises a more aggressive ruleset, including *Repeat* rules, *ArbitraryGenerator* and query size reducing rules, and is intended to escape coverage plateaus over extended time frames and allow control of query size. In this short benchmark, this increased randomness results in a higher invalid query rate (67.7% vs. 51.7%) and lower effective coverage compared to the “Short run” profile. Consequently, the “Short run” profile was selected as the primary benchmark for the evaluation of our fuzzer.

3.4 Performance Evaluation

Performance was evaluated across three levels of encapsulation. Table 3 summarises the throughput in Queries Per Minute (QPM) calculated from the first 10,000 queries (averaged over 20 iterations), along steady-state throughput measured via the Criterion framework. Performance metrics for other profiles can be found in Tables 6 and 7.

Table 3: Performance metrics for the Short configuration (10,000 query burst).

Component	Mean QPM	Steady State (Kelem/s)
Full System (Python)	105,272 ($\pm$ 21,937)	—
Rust Engine (Dry Run)	375,575 ($\pm$ 38,052)	3.2
Generator only (Dry Run)	624,207 ($\pm$ 10,209)	8.72

To evaluate performance over a longer run, we let our fuzzer run for 2 hours on the `Euler cluster`, using the “long run” profile. While this run did not find any new bugs, our fuzzer reached an internally recorded coverage of circa 22.5% and ended with almost 150,000 syntactically and semantically correct queries. The coverage reported by `gcov` when running these against SQLite can be found in Table 5.

4 Discussion and Limitations

4.1 Configuration

During the evaluation of our fuzzer, two discrepancies were identified regarding the SQLite environment. First, although the documentation specifies SQLite 3.39.4 as the target version containing 25 manually injected bugs, the provided binary and source code identify as version 3.39.0. Specifically, the binary located at `/usr/bin/sqlite3-3.39.4` and the source at `/home/test/sqlite3-src` both report version 3.39.0, when being passed the `version` argument.

Secondly, a configuration error was identified with respect to the differential oracle path. We mistakenly set the `oracle_path` to `/usr/bin/sqlite3-3.39.4`, using the buggy target binary to verify its own output. While this setup remained valid for detecting crashes, undefined behaviour and memory safety related bugs, it rendered the detection of logic bugs impossible.

We corrected this configuration shortly before the end of the submission period by redirecting the oracle to the stable reference binary located at `/usr/bin/sqlite3`. Following this correction, the fuzzer successfully identified three distinct logic bugs in the target SQLite version. Longer fuzzing runs using the correct oracle binary could yield more logic bugs, which we did not uncover yet.

4.2 Future and Optimization

Our current implementation is limited to SQL, as mutation rules and corpus handlers assume `sqlparser-rs` ASTs. Future work could involve abstracting these types to support general targets, such as compilers or operating systems. This would require more generalised mutation rules and optimised storage types. To expose an ergonomic API, a rewrite of the public-facing re-exports would be necessary.

The oracle and executors currently exhibit poor performance, flagging many false positives and frequently respawning SQLite subprocesses, as well as doing a lot of slow string processing. A native Rust rewrite of the oracle could improve performance, correctness, and allow for tighter integration with the engine and async frameworks like Tokio. Further gains could be achieved using an AFL-style forkserver for SQLite subprocesses.

Additionally, performance improvements in the engine, such as more fine-grained smart pointer usage and the use of arena allocators for hot loops to reduce allocator pressure could yield significant performance boosts.

The low coverage reported by our fuzzer suggests limitations in the mutation rules or scheduling. It would be interesting to plug in a specialised mutator like Nautilus to benchmark our genetic algorithm against a known baseline and discern whether the problem lies in scheduling, mutation rule diversity or seed corpus diversity.

Further, the schedulers currently act as “black boxes”, and adding telemetry to dump fitness over time could help to clarify their behaviour and help optimise hyperparameters.

Finally, coverage collection is currently strongly coupled with the mutation engine, preventing black-box fuzzing and requiring the use of shared memory. Decoupling these components would allow for easier benchmarking, testing and broader target support. Our current `pc-guard` implementation also conflicts with ASAN when address randomisation is enabled, leading to segfaults at startup. Investigating this bug is a priority, as disabling address randomisation may not always be feasible.

5 Conclusion

We successfully developed a modular fuzzing framework and an execution engine targeting SQLite. By leveraging Abstract Syntax Trees (ASTs) for mutation and data storage, combined with dynamic schedulers and corpus caching, our query generator is capable of achieving throughputs exceeding one million queries per minute using an optimised profile, while maintaining a query validity rate of approximately 0.5.

The effectiveness of the fuzzer was demonstrated by the discovery of three distinct crashes

resulting in undefined behaviour and memory corruption within the instrumented SQLite binary. Basic analysis indicates that these crashes originate from at least two distinct root causes. During our most extensive fuzzing campaign, the system reached approximately 46.3% function coverage and 41.8% line coverage, as measured by `gcov`.

While the current implementation provides a robust and modular foundation for SQL fuzzing, several opportunities for optimisation remain. Specifically, refining the memory allocation schemes within the core engine and improving executor orchestration could yield significant performance gains. Furthermore, enhancing the differential testing oracle would reduce the false-positive rate and improve the granularity of information recorded during crash triage.

This project showcases the efficacy of combining structured mutation rules with adaptive scheduling in evolutionary algorithms to navigate complex, grammar-heavy input spaces. The modular architecture of our framework allows for potential extension to other languages, providing a versatile basis for future fuzzing campaigns.

6 Disclaimers

6.1 LLM Usage

LLMs were used to bounce ideas off, help with debugging tasks, and refine the wording of this report. All generated boilerplate code is annotated as such.

6.2 Code Availability

The full source code for our fuzzer, including evaluation scripts and the data used in this report, is available on GitHub at <https://github.com/lmeller-git/ast-sqlite-fuzzer>.

References

- [1] P. Auer, N. Cesa-Bianchi, and P. Fischer, “Finite-time analysis of the multiarmed bandit problem,” *Machine Learning*, vol. 47, no. 2, pp. 235–256, May 1, 2002, ISSN: 1573-0565. DOI: 10.1023/A:1013689704352.
- [2] C. Aschermann, T. Frassetto, T. Holz, P. Jauernig, A.-R. Sadeghi, and D. Teuchert, “NAUTILUS: Fishing for deep bugs with grammars,” presented at the Network and Distributed System Security Symposium, San Diego, CA: Internet Society, 2019, ISBN: 978-1-891562-55-6. DOI: 10.14722/ndss.2019.23412.
- [3] A. Lipowski and D. Lipowska, “Roulette-wheel selection via stochastic acceptance,” *Physica A*, vol. 391, no. 6, pp. 2193–2196, Dec. 2011, ISSN: 03784371. DOI: 10.1016/j.physa.2011.12.004. arXiv: 1109.3627[cs.NE].

Appendix: Supplementary Data

A Coverage Metrics for Long Run Profile

Table 4: Standardized gcov Metrics: Long Run Profile at 10,000 Queries.

Metric	Coverage %	Items Covered	Total Items
Lines	35.2%	28,278	80,277
Functions	41.0%	1,764	4,301
Branches	27.8%	14,907	53,691

Table 5: Standardized gcov Metrics: Long Run Profile at 2 Hours (150,000 Corpus, $\approx 22.5\%$ Internal Edge Coverage).

Metric	Coverage %	Items Covered	Total Items
Lines	41.8%	33,589	80,277
Functions	46.3%	1,993	4,301
Branches	34.4%	18,427	53,691

B Throughput Benchmarks

Table 6: Extended Rust Engine Throughput (QPM of first 10,000 queries over 20 runs).

Profile	Level	Mean QPM	Std. Dev.
Aggressive	Generator only	585,907	56,637
	Engine	318,550	23,714
Generic	Generator only	1,514,529	29,076
	Engine	942,413	53,182

Table 7: Steady-State Throughput Benchmarks via Criterion (Kelem/s).

Benchmark Target	Generator Only	Full Engine
Short Run Configuration	8.72	3.20
Aggressive Configuration	7.77	4.59
Generic Configuration	22.07	12.23

C SQL Keyword Coverage Data

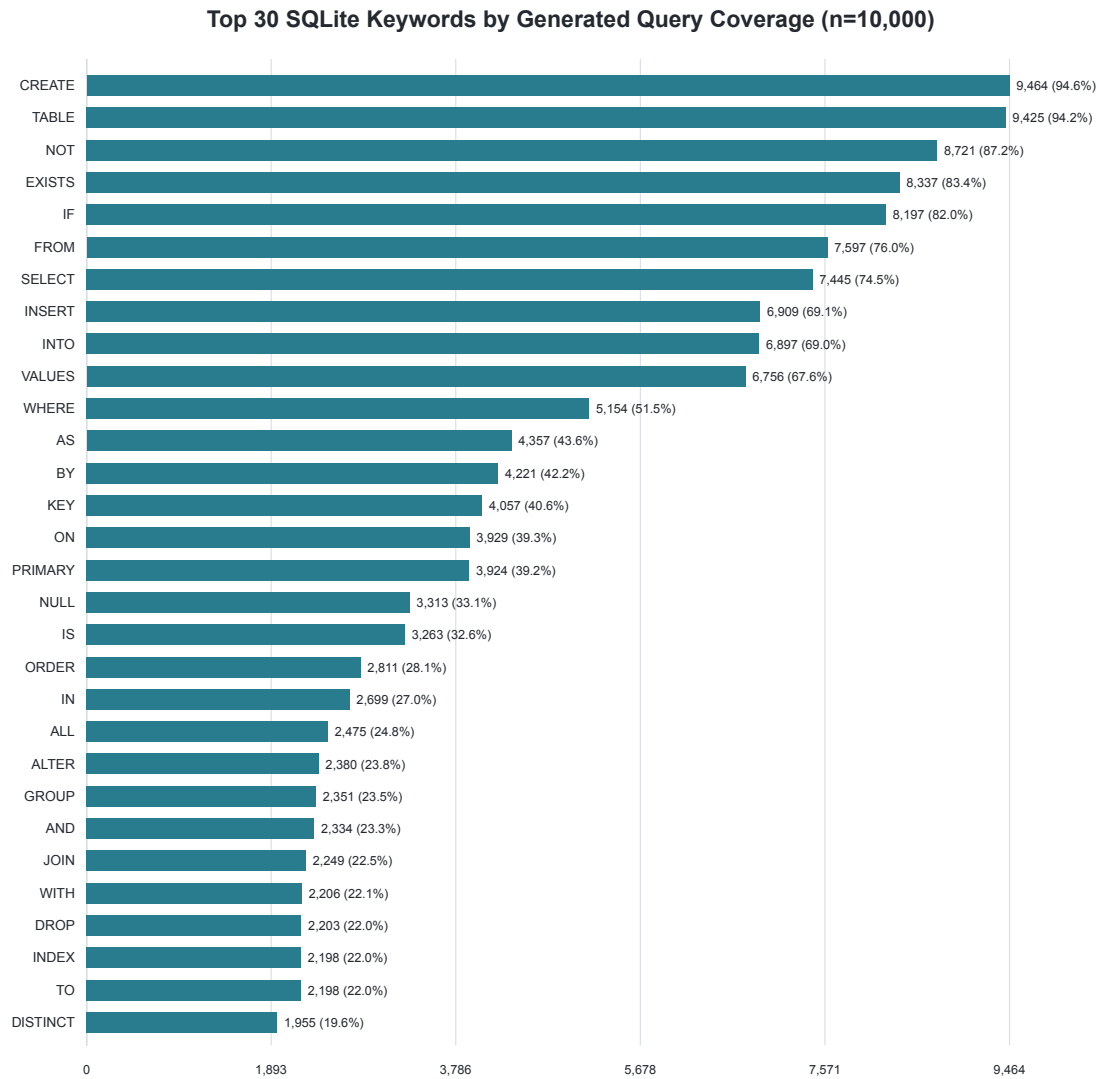


Figure 2: Distribution of the top 30 most frequently generated SQL keywords in the long run profile.

Table 8: SQL keyword coverage across 10,000 queries using the short run configuration.

SQL Keyword	Queries	SQL Keyword	Queries	SQL Keyword	Queries
ABORT	0	EXISTS	8715	OTHERS	0
ACTION	0	EXPLAIN	0	OUTER	0
ADD	1412	FAIL	0	OVER	596
AFTER	459	FILTER	0	PARTITION	368
ALL	1405	FIRST	0	PLAN	0
ALTER	2776	FOLLOWING	0	PRAGMA	1371

Continued on next page

Table 8 – Continued from previous page

SQL Keyword	Queries	SQL Keyword	Queries	SQL Keyword	Queries
ALWAYS	284	FOR	0	PRECEDING	0
ANALYZE	683	FOREIGN	367	PRIMARY	4178
AND	2405	FROM	8250	QUERY	0
AS	8220	FULL	0	RAISE	0
ASC	0	GENERATED	284	RANGE	0
ATTACH	137	GLOB	0	RECURSIVE	722
AUTOINCREMENT	246	GROUP	2165	REFERENCES	886
BEFORE	0	GROUPS	0	REGEXP	0
BEGIN	1342	HAVING	309	REINDEX	0
BETWEEN	288	IF	8654	RELEASE	0
BY	4577	IGNORE	0	RENAME	920
CASCADE	0	IMMEDIATE	0	REPLACE	30
CASE	455	IN	1204	RESTRICT	0
CAST	6840	INDEX	2781	RETURNING	0
CHECK	1355	INDEXED	0	RIGHT	349
COLLATE	583	INITIALLY	0	ROLLBACK	0
COLUMN	1881	INNER	652	ROW	0
COMMIT	383	INSERT	8659	ROWS	0
CONFLICT	0	INSTEAD	0	SAVEPOINT	0
CONSTRAINT	231	INTERSECT	774	SELECT	8006
CREATE	9949	INTO	8541	SET	1280
CROSS	238	IS	1057	TABLE	9955
CURRENT	0	ISNULL	0	TEMP	0
CURRENT_DATE	0	JOIN	2787	TEMPORARY	433
CURRENT_TIME	0	KEY	4415	THEN	455
CURRENT_TIMESTAMP	0	LAST	0	TIES	0
DATABASE	0	LEFT	1422	TO	920
DEFAULT	572	LIKE	656	TRANSACTION	135
DEFERRABLE	0	LIMIT	0	TRIGGER	459
DEFERRED	0	MATCH	0	UNBOUNDED	0
DELETE	550	MATERIALIZED	0	UNION	1662
DESC	256	NATURAL	0	WITH	868
DETACH	0	NO	0	UNIQUE	1126
DISTINCT	1208	NOT	8663	UPDATE	1313
DO	0	NOTHING	0	USING	489
DROP	1784	NOTNULL	0	VACUUM	0
EACH	0	NULL	2191	VALUES	8656
ELSE	276	NULLS	0	VIEW	1398
END	1072	OF	0	VIRTUAL	368
ESCAPE	0	OFFSET	0	WHEN	455
EXCEPT	430	ON	5272	WHERE	6584
EXCLUDE	0	OR	1200	WINDOW	231
EXCLUSIVE	0	ORDER	3333	WITHOUT	1258

Table 9: SQL keyword coverage across 10,000 queries using the long run configuration.

SQL Keyword	Queries	SQL Keyword	Queries	SQL Keyword	Queries
ABORT	134	EXISTS	8337	OTHERS	0
ACTION	13	EXPLAIN	28	OUTER	0
ADD	951	FAIL	9	OVER	673
AFTER	169	FILTER	81	PARTITION	682
ALL	2475	FIRST	472	PLAN	48
ALTER	2380	FOLLOWING	1	PRAGMA	697
ALWAYS	143	FOR	666	PRECEDING	0
ANALYZE	1336	FOREIGN	313	PRIMARY	3924
AND	2334	FROM	7597	QUERY	136
AS	4357	FULL	166	RAISE	120
ASC	106	GENERATED	142	RANGE	92
ATTACH	152	GLOB	0	RECURSIVE	1188
AUTOINCREMENT	184	GROUP	2351	REFERENCES	904
BEFORE	7	GROUPS	0	REGEXP	25
BEGIN	1345	HAVING	53	REINDEX	35
BETWEEN	877	IF	8197	RELEASE	44
BY	4221	IGNORE	47	RENAME	881
CASCADE	228	IMMEDIATE	58	REPLACE	767
CASE	971	IN	2699	RESTRICT	208
CAST	131	INDEX	2198	RETURNING	26
CHECK	793	INDEXED	0	RIGHT	142
COLLATE	1873	INITIALLY	9	ROLLBACK	72
COLUMN	1119	INNER	367	ROW	81
COMMIT	332	INSERT	6909	ROWS	387
CONFLICT	1	INSTEAD	5	SAVEPOINT	208
CONSTRAINT	335	INTERSECT	456	SELECT	7445
CREATE	9464	INTO	6897	SET	1539
CROSS	377	IS	3263	TABLE	9425
CURRENT	43	ISNULL	0	TEMP	12
CURRENT_DATE	0	JOIN	2249	TEMPORARY	803
CURRENT_TIME	0	KEY	4057	THEN	643
CURRENT_TIMESTAMP	0	LAST	139	TIES	334
DATABASE	389	LEFT	977	TO	2198
DEFAULT	454	LIKE	977	TRANSACTION	77
DEFERRABLE	9	LIMIT	542	TRIGGER	317
DEFERRED	12	MATCH	697	UNBOUNDED	0
DELETE	206	MATERIALIZED	67	UNION	962
DESC	350	NATURAL	127	UNIQUE	1512
DETACH	69	NO	34	UPDATE	1121
DISTINCT	1955	NOT	8721	USING	1459
DO	1	NOTHING	1	VACUUM	67
DROP	2203	NOTNULL	0	VALUES	6756
EACH	1	NULL	3313	VIEW	1277
ELSE	369	NULLS	341	VIRTUAL	536
END	1211	OF	864	WHEN	611

Continued on next page

Table 9 – Continued from previous page

SQL Keyword	Queries	SQL Keyword	Queries	SQL Keyword	Queries
ESCAPE	62	OFFSET	239	WHERE	5154
EXCEPT	377	ON	3929	WINDOW	107
EXCLUDE	172	OR	1256	WITH	2206
EXCLUSIVE	20	ORDER	2811	WITHOUT	944

Table 10: Average SQL keyword frequency across 10,000 queries using the short run configuration.

Keyword	Avg. Freq.	Keyword	Avg. Freq.	Keyword	Avg. Freq.
ABORT	0.0	EXISTS	1.9391	OTHERS	0.0
ACTION	0.0	EXPLAIN	0.0	OUTER	0.0
ADD	0.1706	FAIL	0.0	OVER	0.0648
AFTER	0.0463	FILTER	0.0	PARTITION	0.0405
ALL	0.1557	FIRST	0.0	PLAN	0.0
ALTER	0.3274	FOLLOWING	0.0	PRAGMA	0.1516
ALWAYS	0.0286	FOR	0.0	PRECEDING	0.0
ANALYZE	0.1218	FOREIGN	0.047	PRIMARY	0.6303
AND	0.4257	FROM	2.966	QUERY	0.0
AS	7.6718	FULL	0.0	RAISE	0.0
ASC	0.0	GENERATED	0.0286	RANGE	0.0
ATTACH	0.0137	GLOB	0.0	RECURSIVE	0.0779
AUTOINCREMENT	0.0465	GROUP	0.3926	REFERENCES	0.103
BEFORE	0.0	GROUPS	0.0	REGEXP	0.0
BEGIN	0.135	HAVING	0.0317	REINDEX	0.0
BETWEEN	0.0375	IF	1.8831	RELEASE	0.0
BY	1.0161	IGNORE	0.0	RENAME	0.0931
CASCADE	0.0	IMMEDIATE	0.0	REPLACE	0.003
CASE	0.0481	IN	0.2615	RESTRICT	0.0
CAST	6.5828	INDEX	0.4166	RETURNING	0.0
CHECK	0.1575	INDEXED	0.0	RIGHT	0.0406
COLLATE	0.0924	INITIALLY	0.0	ROLLBACK	0.0
COLUMN	0.2227	INNER	0.0704	ROW	0.0
COMMIT	0.0385	INSERT	2.2093	ROWS	0.0
CONFLICT	0.0	INSTEAD	0.0	SAVEPOINT	0.0
CONSTRAINT	0.0284	INTERSECT	0.1318	SELECT	3.0163
CREATE	2.7224	INTO	2.1946	SET	0.1733
CROSS	0.0244	IS	0.2842	TABLE	2.5394
CURRENT	0.0	ISNULL	0.0	TEMP	0.0
CURRENT_DATE	0.0	JOIN	0.4298	TEMPORARY	0.0434
CURRENT_TIME	0.0	KEY	0.6773	THEN	0.0871
CURRENT_TIMESTAMP	0.0	LAST	0.0	TIES	0.0
DATABASE	0.0	LEFT	0.1779	TO	0.0931

Continued on next page

Table 10 – Continued from previous page

Keyword	Avg. Freq.	Keyword	Avg. Freq.	Keyword	Avg. Freq.
DEFAULT	0.073	LIKE	0.1647	TRANSACTION	0.0136
DEFERRABLE	0.0	LIMIT	0.0	TRIGGER	0.0463
DEFERRED	0.0	MATCH	0.0	UNBOUNDED	0.0
DELETE	0.0581	MATERIALIZED	0.0	UNION	0.1923
DESC	0.0256	NATURAL	0.0	WITH	0.1071
DETACH	0.0	NO	0.0	UNIQUE	0.1422
DISTINCT	0.2032	NOT	2.1306	UPDATE	0.2049
DO	0.0	NOTHING	0.0	USING	0.0534
DROP	0.2003	NOTNULL	0.0	VACUUM	0.0
EACH	0.0	NULL	0.5558	VALUES	2.1585
ELSE	0.0286	NULLS	0.0	VIEW	0.1841
END	0.1103	OF	0.0	VIRTUAL	0.0396
ESCAPE	0.0	OFFSET	0.0	WHEN	0.0871
EXCEPT	0.0631	ON	0.8057	WHERE	1.6084
EXCLUDE	0.0	OR	0.2599	WINDOW	0.0243
EXCLUSIVE	0.0	ORDER	0.583	WITHOUT	0.1743

Table 11: Average SQL keyword frequency across 10,000 queries using the long run configuration.

SQL Keyword	Avg. Freq.	SQL Keyword	Avg. Freq.	SQL Keyword	Avg. Freq.
ABORT	0.0134	EXISTS	1.9104	OTHERS	0.0
ACTION	0.0013	EXPLAIN	0.0029	OUTER	0.0
ADD	0.1215	FAIL	0.0009	OVER	0.072
AFTER	0.0169	FILTER	0.0082	PARTITION	0.0736
ALL	0.5017	FIRST	0.0519	PLAN	0.0048
ALTER	0.3002	FOLLOWING	0.0001	PRAGMA	0.0732
ALWAYS	0.0144	FOR	0.0947	PRECEDING	0.0
ANALYZE	0.1807	FOREIGN	0.0384	PRIMARY	0.58
AND	0.3502	FROM	2.215	QUERY	0.0136
AS	1.0033	FULL	0.0169	RAISE	0.0125
ASC	0.0107	GENERATED	0.0143	RANGE	0.01
ATTACH	0.0154	GLOB	0.0	RECURSIVE	0.1386
AUTOINCREMENT	0.0251	GROUP	0.4044	REFERENCES	0.1044
BEFORE	0.0007	GROUPS	0.0	REGEXP	0.0026
BEGIN	0.1396	HAVING	0.0053	REINDEX	0.0035
BETWEEN	0.1108	IF	1.8216	RELEASE	0.0044
BY	0.8489	IGNORE	0.0049	RENAME	0.0917
CASCADE	0.0237	IMMEDIATE	0.058	REPLACE	0.0846
CASE	0.1111	IN	0.5148	RESTRICT	0.0211
CAST	0.0144	INDEX	0.3607	RETURNING	0.0026
CHECK	0.0873	INDEXED	0.0	RIGHT	0.0172

Continued on next page

Table 11 – Continued from previous page

SQL Keyword	Avg. Freq.	SQL Keyword	Avg. Freq.	SQL Keyword	Avg. Freq.
COLLATE	0.3075	INITIALLY	0.0009	ROLLBACK	0.0072
COLUMN	0.14	INNER	0.0375	ROW	0.0099
COMMIT	0.034	INSERT	1.3982	ROWS	0.0402
CONFLICT	0.0001	INSTEAD	0.0005	SAVEPOINT	0.0211
CONSTRAINT	0.0386	INTERSECT	0.0644	SELECT	2.1608
CREATE	2.5676	INTO	1.4287	SET	0.2068
CROSS	0.0381	IS	1.0971	TABLE	2.3372
CURRENT	0.0044	ISNULL	0.0	TEMP	0.0112
CURRENT_DATE	0.0	JOIN	0.3001	TEMPORARY	0.0879
CURRENT_TIME	0.0	KEY	0.6141	THEN	0.0925
CURRENT_TIMESTAMP	0.0	LAST	0.0143	TIES	0.0342
DATABASE	0.0404	LEFT	0.1168	TO	0.3368
DEFAULT	0.0647	LIKE	0.1451	TRANSACTION	0.0077
DEFERRABLE	0.0009	LIMIT	0.059	TRIGGER	0.0319
DEFERRED	0.0012	MATCH	0.0817	UNBOUNDED	0.0
DELETE	0.0224	MATERIALIZED	0.0068	UNION	0.1128
DESC	0.0358	NATURAL	0.0127	UNIQUE	0.2078
DETACH	0.0069	NO	0.0037	UPDATE	0.1381
DISTINCT	0.3277	NOT	2.5692	USING	0.1742
DO	0.0001	NOTHING	0.0001	VACUUM	0.0067
DROP	0.2655	NOTNULL	0.0	VALUES	1.3635
EACH	0.0001	NULL	0.6817	VIEW	0.1711
ELSE	0.0377	NULLS	0.0356	VIRTUAL	0.0541
END	0.138	OF	0.1296	WHEN	0.0883
ESCAPE	0.0065	OFFSET	0.0264	WHERE	1.0501
EXCEPT	0.0458	ON	0.6339	WINDOW	0.0109
EXCLUDE	0.0177	OR	0.1665	WITH	0.3684
EXCLUSIVE	0.002	ORDER	0.3728	WITHOUT	0.1004